Assignment #5 — Strategy Pattern Report

Course: ShP-2216 – Software Design Patterns
Academic Year: 2026–2027 (2nd year, 4th trimester)
Institution: Astana IT University – School of SE
Student: Dias
GitHub Repository: https://github.com/HumbleCherry10/SDP-5

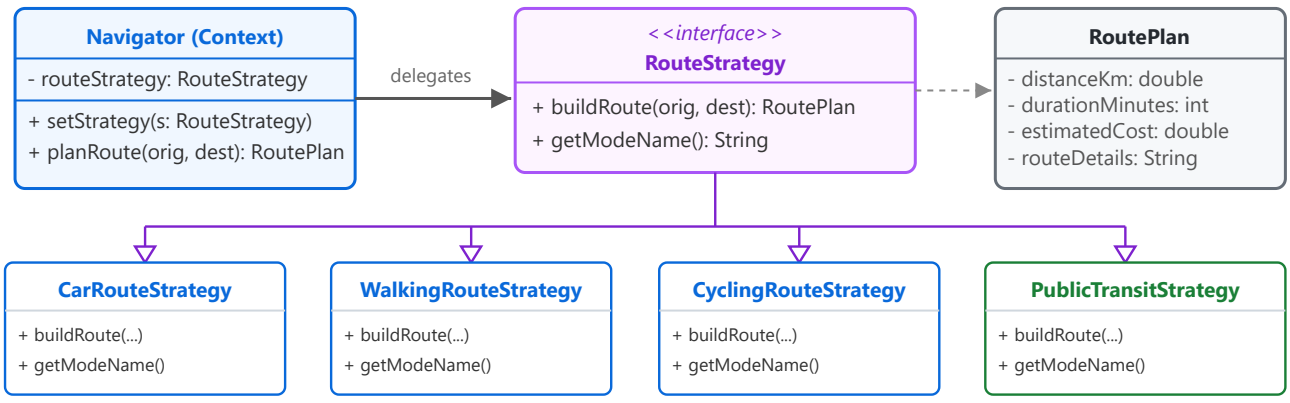
1. Introduction

For this assignment, **Option A — Navigation / Route Planning** was selected. In modern geospatial navigation systems (e.g., Google Maps, 2GIS), users calculate journeys between origins and destinations across radically different modal constraints:

- **Car / Driving:** Optimizes for highways and arterial bypasses, factors in road traffic congestion, and estimates fuel and toll expenses.
- **Walking / Pedestrian:** Routes through sidewalks, urban parks, footbridges, and pedestrian-only zones while strictly avoiding vehicular highways. Incurs zero financial cost.
- **Bicycle / Cycling:** Prioritizes dedicated cycling paths, greenways, and minimal slope gradients for commuter safety and stamina.
- **Public Transit:** Coordinates multi-modal connections (metro, buses), fixed schedule stops, transfers, and flat tariff ticketing.

The **Strategy Design Pattern** solves this problem by encapsulating each distinct route-planning algorithm behind a common interface (`RouteStrategy`). A lightweight context (`Navigator`) maintains a reference to the active strategy and delegates execution directly to it, granting the client dynamic runtime strategy switching without coupling or complex conditionals.

2. UML Class Diagram



3. Clean Code Section

The solution strictly adheres to Clean Code standards (Robert C. Martin, Ch. 10):

1. Elimination of Conditional Logic in Favor of Polymorphism

Instead of hardcoding brittle `if-else` or `switch` cascades in `Navigator`, algorithms are dispatched polymorphically at runtime via the `RouteStrategy` interface.

```
// BEFORE (Fragile branching antipattern):
if (mode.equals("CAR")) { routeCar(...); }
else if (mode.equals("WALK")) { routeWalk(...); }

// AFTER (Clean polymorphic dispatch in Navigator.java):
public RoutePlan planRoute(String origin, String destination) {
    return routeStrategy.buildRoute(origin, destination); // 0 conditionals!
}
```

2. Meaningful, Intention-Revealing Names

Identifiers accurately communicate business intent (`CarRouteStrategy`, `WalkingRouteStrategy`, `buildRoute`, `RoutePlan`), eliminating guesswork and misleading naming.

```
public interface RouteStrategy {
    RoutePlan buildRoute(String origin, String destination);
    String getModeName();
}
```

3. Single Responsibility Principle (SRP)

Each concrete strategy encapsulates only the rules, speeds, and calculations for its specific transit mode. `Navigator` is solely responsible for holding the reference and executing delegation.

4. Context Remains Algorithm-Agnostic (Dependency Inversion)

`Navigator` depends purely on the abstract interface `RouteStrategy`. It possesses zero coupling to speeds, toll fees, highway maps, or pedestrian walking steps.

```
public class Navigator {
    private RouteStrategy routeStrategy;
    public void setStrategy(RouteStrategy routeStrategy) {
        this.routeStrategy = Objects.requireNonNull(routeStrategy, "Strategy cannot be null");
    }
}
```

5. Easy Extensibility (Open/Closed Principle)

We demonstrated open extensibility by implementing `PublicTransitRouteStrategy` without modifying or recompiling `Navigator` or existing strategies.

4. Strategy vs. State

While **Strategy** and **State** share identical structural class diagrams, their fundamental intent and lifecycle control are opposite:

- **Client Control vs. Autonomous Internal Transitions:** In the *Strategy Pattern*, the client externally selects and configures the algorithm suitable for its objective (e.g., user selects Driving vs. Walking). Individual strategies are unaware of one another. In contrast, the *State Pattern* manages an internal state lifecycle where transitions trigger automatically based on context state changes (e.g., Draft → Moderation → Published).
- **Why Navigation Calls for Strategy:** A navigation engine does not undergo internal life-cycle state changes; walking, driving, and cycling are purely interchangeable algorithms selected by the user to compute a path. Strategy is therefore the precise and correct pattern.

5. Conclusion

The Strategy pattern decoupled algorithm selection from route execution. The minor overhead of introducing interface and concrete strategy classes is vastly outweighed by the elimination of regressions, full adherence to SOLID principles, and frictionless extensibility.

6. Link to GitHub Repository

The complete compiling source code and Git commit history are available at:
https://github.com/HumbleCherry10/SDP-5